S.S.V.P.S's B. S. Deore Polytechnic, Dhule
Department of Computer Engineering
Academic Year : 2026-2027

SEMINAR AND PROJECT INITIATION COURSE

Sem : CO 5 K	SPI – 315003	Date :

Topic : Malicious URL Detection System to protect user from phishing attacks

Roll No.	Name of Student -
23	Ujwal Sandeep Chitte
51	Abhijit Sunil Patil
58	Janhavi Krishna Patil
63	Kimaya Nitin Patil

Contents

- Abstract

1. Introduction
2. Literature Review
3. Information to Seminar Topic
4. Advantages and Disadvantages
5. Conclusion
6. Project Initiation :
   - 6.1 Description of Problem Statement
   - 6.2 Scope and Objectives
   - 6.3 Stakeholder
   - 6.4 Platform/ Equipment/ Resources identification
7. Bibliography
8. References

Abstract

Malicious URLs, especially phishing links shared through WhatsApp, SMS, and email, are a common way for attackers to steal credentials and money, and most users cannot tell them apart from genuine links. Blacklist-based protection reacts only after a link is reported, so new malicious domains often go undetected. This seminar proposes a Malicious URL Detection System that uses a machine learning classifier (such as Random Forest), trained on URL features like length, special characters, IP usage, and domain age, along with a threat-intelligence cross-check. Users paste a link and get a Safe, Suspicious, or Malicious verdict through a simple web interface. The system is evaluated using accuracy, precision, recall, and F1-score.

1. Introduction

- The internet is used daily for banking, shopping, education, and communication, and the URL is the main way users reach any website.
- A malicious URL is a link that leads to harmful content, such as a fake login page, a malware download, a fraudulent redirect, or a defaced website.
- With the rapid growth of UPI payments and online banking in India, these attacks have become more damaging.
- Attackers send links that imitate banks, delivery companies, government portals, and payment apps through WhatsApp, SMS, and email.
- Messages often look urgent or arrive from a known contact, so many users click without checking.
- A single click can lead to stolen credentials, leaked OTPs, financial loss, or a malware infection.
- Traditional protection relies on blacklists of already-reported sites, which are reactive, so a new malicious domain can stay active for hours or days before it is blocked.

2. Literature Survey

Sr. No.	Approach Type	Technologies Used	Dataset	Key Objective	Limitations
1	Machine-learning-based review	Machine learning, URL feature extraction, classification algorithms	Multiple datasets reviewed in the paper	Review existing malicious URL detection techniques and identify research gaps.	Performance varies across datasets; newly created malicious URLs remain challenging.
2	Deep-learning-based detection (URLNet)	CNNs, character-level and word-level URL representations	Labelled malicious and legitimate URLs	Automatically learn URL patterns and classify URLs as malicious or legitimate.	Requires sufficient labelled data and computational resources; unseen attack patterns may reduce performance.
3	Comparative machine-learning approach	Conventional machine-learning and quantum machine-learning methods	Experimental datasets used in the study	Compare approaches for malicious URL detection in the studied setting.	Findings depend on dataset quality, experimental settings and model scalability.

Sr. No.	Approach Type	Technologies Used	Dataset	Key Objective	Limitations
4	Risk-adaptive browser prompt filtering	TypeScript, WebExtensions API, ONNX Runtime Web, regex, token classifier	Proposed precision, recall, F1, leakage and utility metrics	Score risk and allow, warn, redact or replace prompt content.	Browser coverage limited; results depend on detection and policy quality.
5	Client-side representation transformation	Python, transformer encoder, PyTorch, inference server	No fixed dataset; leakage, utility, bandwidth and latency tests proposed	Perturb prompt representations before compatible cloud inference.	May leak data or harm quality; requires server-side adaptation.
6	Prompt-threat detection with minimal logging	Python, FastAPI, text classifier, hashes, vector similarity	No fixed dataset; evaluation metrics proposed	Detect prompt injection and sensitive-data exposure with minimal logs.	Fingerprints may leak data or mismatch; detection alone is insufficient.

Sr. No.	Approach Type	Technologies Used	Dataset	Key Objective	Limitations
7	Enterprise API gateway and anomaly detection	OAuth 2.0, JWT, ML anomaly detection	Banking, healthcare, IoT; benchmark not specified	Secure API access and detect unauthorized activity or DDoS.	Adds latency; may produce errors; does not inspect PII in request content.
8	Federated learning with secure gateway	Kong Gateway, OAuth 2.0, JWT, TLS, differential privacy, zero-trust	CIFAR-10, MIMIC-III; 100 simulated edge nodes	Protect federated learning while keeping raw data local.	Privacy adds accuracy, communication and latency costs; scalability needs testing.
9	Sensitive-data replacement	Tokenization, placeholders, reversible mapping	Dataset details not fully visible in supplied image	Replace sensitive values before sending prompts to external models.	Encoding may be exposed or context may be lost if mapping is mishandled.

Sr. No.	Approach Type	Technologies Used	Dataset	Key Objective	Limitations
10	Context-aware PII detection	NER, contextual classifiers, policy rules	Not visible in supplied image	Identify which personal data is relevant to the user's request.	False positives and missed entities can reduce utility and privacy.
11	Prompt-injection and data-leakage screening	Input classifiers, pattern rules, security filters	Not visible in supplied image	Detect unsafe instructions and possible disclosure before model processing.	Evasion and new attack patterns can bypass static filters.
12	Privacy-preserving LLM pipeline	Local preprocessing, redaction, secure API, audit controls	Not visible in supplied image	Combine detection, redaction and controlled model access in one workflow.	Extra processing can increase latency; needs broad real-world evaluation.

3. Information to seminar topic

1. Structure of a URL

- A URL contains protocol, domain name, extension, path, and parameters.
- Attackers use fake or similar-looking domains to trick users.

2. Types of Malicious URLs

- Phishing URLs – Steal passwords and personal details.
- Malware URLs – Download harmful software.
- Defacement URLs – Show modified or hacked websites.
- Spam URLs – Lead to unwanted or fraudulent websites.

3. Common Evasion Techniques

- Fake or misspelled domain names.
- Shortened URLs and long, complex links.
- Newly registered domains and misleading HTTPS links.

3. Information to seminar topic

4. Feature Extraction

- Lexical Features: URL length, dots, numbers, symbols, and keywords.
- Host-based Features: Domain age, DNS records, and registration details.

5. Machine Learning Classification

- Collect and clean URL datasets.
- Extract features and train ML models.
- Use Random Forest, Decision Tree, or SVM.

6. Datasets and Tools

- Datasets: UCI, Kaggle, and PhishTank.
- Tools: Python, Pandas, Scikit-learn, and Flask/Streamlit.

3. Information to seminar topic

7. Proposed System Architecture

- URL Input → Pre-processing → Feature Extraction → ML Model → Blacklist Check → Final Result.
- Output: Safe, Suspicious, or Malicious.

8. Performance Evaluation

- Accuracy – Overall correct predictions.
- Precision – Correct malicious URL predictions.
- Recall – Malicious URLs detected.
- F1-score – Balance between precision and recall.

9. Legal and Ethical Aspects

- Follow applicable cybercrime and data protection laws.
- Protect user privacy and use the system only for defensive purposes.

4. Advantages / Disadvantages

Advantages

- **Detects new attack**s: It can detect new malicious URLs that are not in the blacklist.
- **Fast results:** It checks URLs within seconds and is easy to use.
- **Safe checking:** It checks the URL without opening the harmful website.
- **Low cost:** It uses free tools and public datasets.

Disadvantages

- **Dataset dependency:** Accuracy depends on the quality of the training data.
- **Wrong results:** It may mark a safe URL as harmful or miss a harmful URL.
- **Limited detection:** Some specially designed malicious URLs may escape detection.
- **Slow checking:** Checking domain information may take extra time.

5. Conclusion

- Malicious URLs and phishing links are used to steal personal information, passwords, and money. Traditional methods may not detect new threats. Machine Learning helps identify malicious URLs by checking their features and patterns.

- The proposed system allows users to check suspicious links quickly and easily before clicking them. Combining Machine Learning with a blacklist can improve detection. In the future, the system can be improved by adding browser extensions, WhatsApp or Telegram bots, QR-code scanning, and regular updates to detect new phishing attacks and improve online safety.

6. Project Initiation

- Identify the problem of phishing attacks through malicious URLs.
- Understand the need for detecting fake and suspicious websites.
- Develop a system to analyze URLs and detect potential threats.
- Classify URLs as legitimate or malicious.
- Warn users about potentially dangerous links.
- Aim to protect users’ personal information and improve online safety.

6.1 Problem Statement

- Malicious URLs are used in phishing attacks to steal passwords, banking details, and personal information.
- These links are shared through emails, SMS, and messaging applications.
- Traditional blacklist methods may fail to detect newly created malicious URLs.
- Therefore, a system is needed to analyze URL features using machine learning and threat intelligence.
- The system aims to identify malicious URLs and warn users about potential phishing threats.

6.2 Scope and Objectives

Scope:

- Detect potentially malicious and legitimate URLs.
- Extract URL features and analyze suspicious links.
- Use machine learning and threat-intelligence sources.
- Display detection results through a simple web interface.

Objectives:

- To study phishing attacks and malicious URLs.
- To identify important URL features.
- To study machine-learning algorithms for URL classification.
- To check known threats using threat-intelligence sources.
- To display clear detection results through a web interface.
- To create awareness about phishing and online security.

6.3 Stake Holder

- **Internet Users:** Check suspicious URLs before opening them.
- **Students and Researchers:** Study malicious URL detection techniques.
- **Cybersecurity Professionals:** Use detection results for security support.
- **Educational Institutions:** Promote cybersecurity awareness.
- **Organisations:** Screen suspicious links for potential threats.
- **Project Team:** Design, develop, and evaluate the system.

6.4 Platform / Equipment / Resources identification

- **Computer or Laptop:** For research and development.
- **Internet Connection:** To access research papers and datasets.
- **Python:** For machine-learning development.
- **Pandas:** For data processing and analysis.
- **Scikit-learn:** For training and evaluating machine-learning models.
- **Flask or Streamlit:** For creating the web interface.
- **Google Colab or Jupyter Notebook:** For model training and experimentation.
- **PhiUSIIL Dataset:** For training and testing URL classification models.
- **PhishTank / Google Safe Browsing:** For checking known malicious URLs.

Bibliography

- Google for Developers, “Google Safe Browsing API.” Available at: https://developers.google.com/safe-browsing/reference
- Scikit-learn, “Machine Learning in Python.” Available at: https://scikit-learn.org/
- Pandas, “Python Data Analysis Library.” Available at: https://pandas.pydata.org/
- Streamlit, “Build Data Apps.” Available at: https://docs.streamlit.io/
- Flask, “Flask Web Development Framework.” Available at: https://flask.palletsprojects.com/
- PhiUSIIL Phishing URL Dataset, “Phishing URL Classification Dataset.” Available at: https://archive.ics.uci.edu/

References

- M. Aljabri et al., “Detecting Malicious URLs Using Machine Learning Techniques: Review and Research Directions,” IEEE Access, vol. 10, pp. 121395–121417, 2022. https://doi.org/10.1109/ACCESS.2022.3222307
- H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, “URLNet: Learning a URL Representation with Deep Learning for Malicious URL Detection,” Proceedings of the 2018 ACM International Conference on Web Search and Data Mining, 2018. https://arxiv.org/abs/1802.03162
- N. Reyes-Dorta, P. Caballero-Gil, and C. Rosa-Remedios, “Detection of Malicious URLs Using Machine Learning,” Wireless Networks, vol. 30, pp. 7543–7560, 2024. https://doi.org/10.1007/s11276-024-03700-w
- D. Sahoo, C. Liu, and S. C. H. Hoi, “Malicious URL Detection Using Machine Learning: A Survey,” arXiv preprint arXiv:1701.07179, 2017. https://arxiv.org/abs/1701.07179
- F. Vanhoenshoven, G. Nápoles, R. Falcon, K. Vanhoof, and M. Köppen, “Detecting Malicious URLs Using Machine Learning Techniques,” IEEE Symposium Series on Computational Intelligence, pp. 1–8, 2016.
- O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine Learning Based Phishing Detection from URLs,” Expert Systems with Applications, vol. 117, pp. 345–357, 2019. https://doi.org/10.1016/j.eswa.2018.09.029

References

- J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond Blacklists: Learning to Detect Malicious Web Sites from Suspicious URLs,” Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1245–1254, 2009.
- S. Garera, N. Provos, M. Chew, and A. D. Rubin, “A Framework for Detection and Measurement of Phishing Attacks,” Proceedings of the ACM Workshop on Recurring Malcode, pp. 1–8, 2007.
- A. Le, A. Markopoulou, and M. Faloutsos, “PhishDef: URL Names Say It All,” Proceedings of IEEE INFOCOM, pp. 191–195, 2011.
- R. Verma and K. Dyer, “On the Character of Phishing URLs: Accurate and Robust Statistical Learning Classifiers,” Proceedings of the 5th ACM Conference on Data and Application Security and Privacy, pp. 111–122, 2015.
- F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html
- T. M. Mitchell, Machine Learning. McGraw-Hill, 1997.
- A. C. Müller and S. Guido, Introduction to Machine Learning with Python: A Guide for Data Scientists. O’Reilly Media, 2016.

Thank you